

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import StandardScaler
import seaborn as sns
```

```
= make_classification(
n_ samples=1000, n
features=7 , n
informative=5, n
redundant=2, n
classes=3, random
state=42

df = pd . DataFrame(X, columns=[
" Age" ,
"Annual Income" ,
"Spending_Score" ,
"Visit _ Frequency" ,
"Membership_Years " ,
"Online_Engagement_Score " ,
"Previous_Offer_Response"
I)

" Age ] = (df["Age"] - df["Age"].min()) / (df["Age"].max() - df["Age"].min())
"Age " ] = (df["Age"] * 42 + 18) . astype(int)

"Annual_Income"] = (df["Annual_Income"] - df["Annual_Income"].min()) / (df["Annual_Income"].max() - df["Annual_Income"].min())
"Annual_Income"] = (df["Annual_Income"] * 130000 + 20000) . astype(int)

"Spending_Score"] = (df["Spending_Score"] - df["Spending_Score"].min()) / (df["Spending_Score"].max() - df["Spending_Score"].min())
"Spending_Score"] = (df["Spending_Score"] * 99 + 1) . astype(int)

"Visit_Frequency"] = (df["Visit_Frequency"] - df["Visit_Frequency"].min()) / (df["Visit_Frequency"].max() - df["Visit_Frequency"].min())
"Visit_Frequency"] = (df["Visit_Frequency"] * 19 + 1) . astype(int)

"Membership_Years"] = (df["Membership_Years"] - df["Membership_Years"].min()) / (df["Membership_Years"].max() - df["Membership_Years"].min())
"Membership_Years"] = (df["Membership_Years"] * 99 + 1) . astype(int)

"Online_Engagement_Score"] = (df["Online_Engagement_Score"] - df["Online_Engagement_Score"].min()) / (df["Online_Engagement_Score"].max() - df["Online_Engagement_Score"].min())
"Online_Engagement_Score"] = (df["Online_Engagement_Score"] * 99 + 1) . astype(int)

"Previous_Offer_Response"] = (df["Previous_Offer_Response"] - df["Previous_Offer_Response"].min()) / (df["Previous_Offer_Response"].max() - df["Previous_Offer_Response"].min())
"Previous_Offer_Response"] = (df["Previous_Offer_Response"] * 99 + 1) . astype(int)

"Customer_Response"] = (df["Customer_Response"] - df["Customer_Response"].min()) / (df["Customer_Response"].max() - df["Customer_Response"].min())
"Customer_Response"] = (df["Customer_Response"] * 99 + 1) . astype(int)

df. head()
```

Age	Annual_Income	Spending_Score	Visit_Frequency	Membership_Years	Online_Engagement_Score	Previous_Offer_Response	Customer_Response
18	130000	1	1	1	1	1	1

```
o 30 51216 32 11 5 42 1 27 110419 72 10 3 63 1
```

```
2 43 79825 26 9 6 73
```

```
3 48 78818 68 7 4 62
```

```
4 32 57581 26 10 5 55
```

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df = df.drop_duplicates()
```

```
plt.subplot(2, 3, 1) plt .  
boxplot(df[ "Age" ] )  
plt.title("Age")
```

```
plt.subplot(2, 3, 2)  
plt . boxplot (df[ Annual_Income" ] ) plt.title(  
"Annual Income")
```

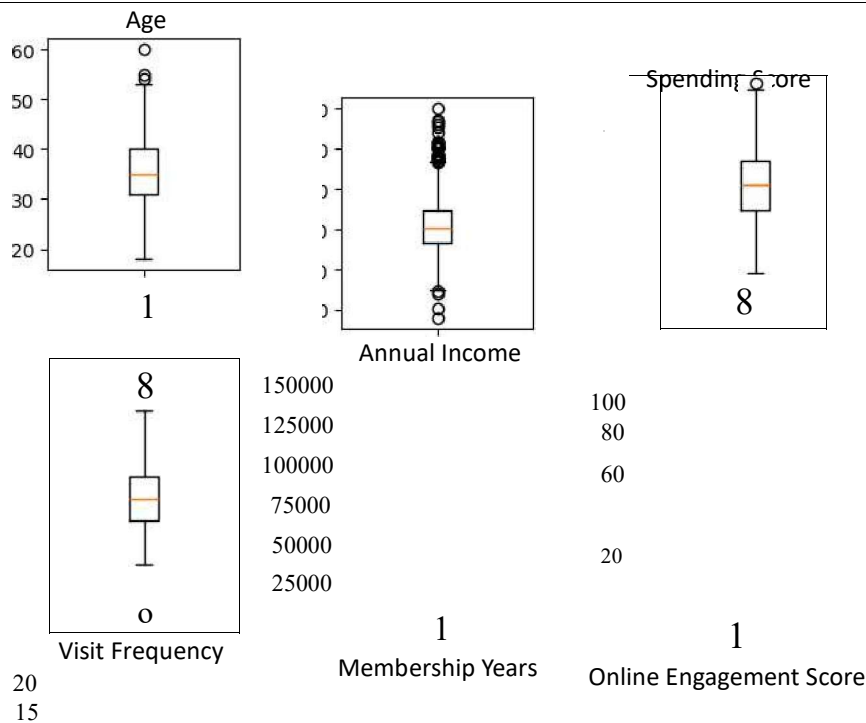
```
plt.subplot(2, 3, 3) plt . boxplot (df[  
"Spending_Score" ] )  
plt.title("Spending Score")
```

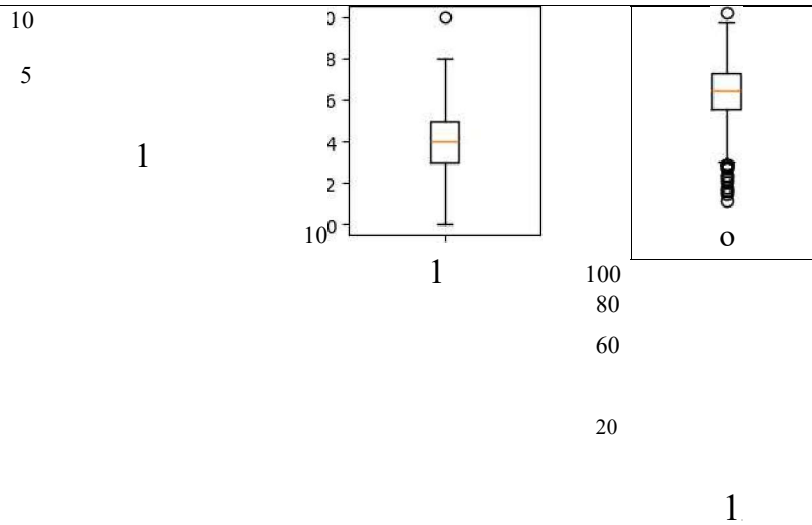
```
plt.subplot(2, 3, 4) plt . boxplot (df[  
"Visit_Frequency" ] ) plt.title("Visit  
Frequency")
```

```
plt.subplot(2, 3, 5) plt . boxplot (df[  
"Membership_Years" ] )  
plt.title("Membership Years")
```

```
plt.subplot(2, 3, 6) plt . boxplot (df[ "Online_  
Engagement_Score" ] ) plt.title("Online  
Engagement Score")
```

```
plt . tight_layout  
( ) plt. show()
```





```
def remove_outliers_iqr(data, column) :
    Q1 = data[column] . quantile(0.25)
    Q3 = data[column] . quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[column] >= lower) & (data[column] <=
upper)]

df = remove_outliers_iqr(df, "Annual_Income")
print(df.shape)
```

(963, 8)

```
import seaborn as sns

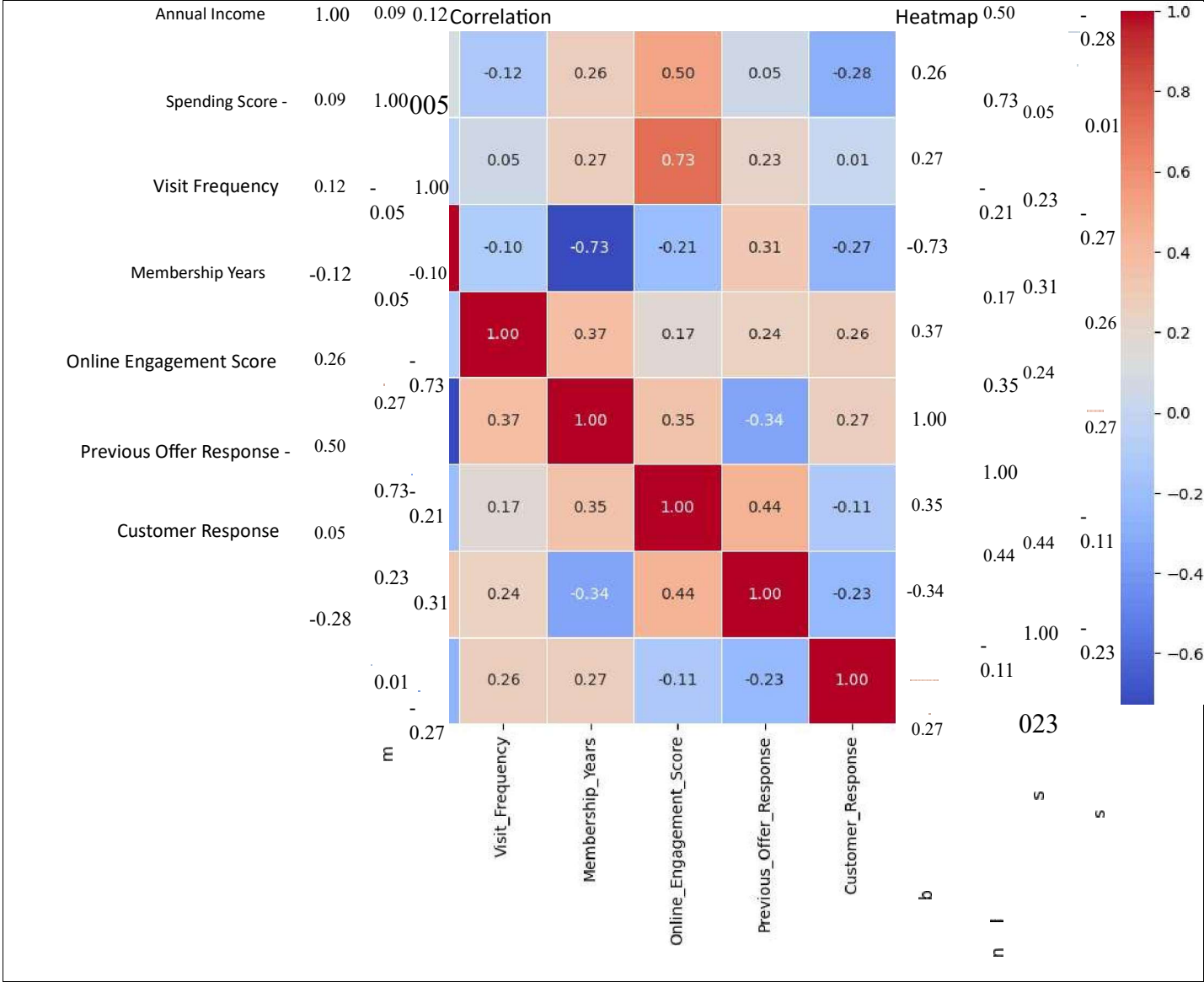
plt.figure(figsize=(10, 8))

correlation_matrix = df.corr()

sns.heatmap(
    correlation_matrix,
```

```
    annot=True,
    fmt=" ",
    cmap= "
coolwarm",
    linewidths=0.5

plt.title("Correlation
Heatmap") plt . show( )
```



```
def accuracy(TP, TN, FP, FN):
    return (TP + TN) / (TP + TN + FP + FN + 1e-9)

def precision(TP, FP):
    return TP / (TP + FP + 1e-9)

def recall(TP, FN):
    return TP / (TP + FN + 1e-9)

def f1(TP, FP, FN):
    p = precision(TP, FP)
    r = recall(TP, FN)
    return 2 * (p * r) / (p + r + 1e-9)
```

```
A = df[["Online_Engagement_Score"]]
B = df["Customer_Response"]

A_train, A_test, B_train, B_test = train_test_split(
    A, B, test_size=0.3, random_state=42
)

scaler_A = StandardScaler()
A_train = scaler_A.fit_transform(A_train)
A_test = scaler_A.transform(A_test)

model_single = LogisticRegression(max_iter=5000)
model_single.fit(A_train, B_train)

B_pred = model_single.predict(A_test)

print("Single Feature Model Trained (A, B)
Single Feature Model Trained (A, B)
```

```
X = df[["Online_Engagement_Score", "Annual_Income"]]
Y = df["Customer_Response"]

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

scaler_X = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)

model_multi = LogisticRegression(max_iter=5000)
model_multi.fit(X_train, Y_train)

Y_pred = model_multi.predict(X_test)

print("Multi Feature Model Trained (X, Y)
Multi Feature Model Trained (X, Y)
```

```
TP = 18
TN = 140
```

```

cm_single = confusion_matrix(
    (B_test == 2).astype(int),
    (B_pred == 2).astype(int)

cm_multi = confusion_matrix(
    (Y_test == 2).astype(int),
    (Y_pred == 2).astype(int)

print("Binary Confusion Matrix - Single Feature Model" ) print(cm_single)

print("\n- Multi Feature Model") print(cm_multi)

```

Binary Confusion Matrix - Single Feature Model

```

[ [128 65]
  [ 56 40] ]

```

- Multi Feature Model

```

[ [140 53]
  [ 78 18] ]

```

```

TP_s = 40
TN_s = 128

FP_s = 65
FN_s = 56

```

```

print("Single Feature Model Metrics" ) print(
"Accuracy : " TN_s, FP_s, ", precision(TP_s,
FP_s)) print( "Recall : " , recall(TP_s, FN_s))
print("F1 Score: " , f1(TP_s,FP_s, FN_s))

```

Single Feature Model Metrics

Accuracy: 0.5813148788907221 Precision:
0.38095238094875283
Recall: 0.4166666666623264
F1 Score: 0.3980099497457984

```

FP = 53
FN = 78

```

```

print("Single Feature Model Metrics" ) print( "Accuracy : " ,
accuracy(TP, TN, FP, print("Precision:", precision(TP, FP)) print( "Recall
: " , recall(TP, FN)) print("F1 Score: " , f1(TP, FP, FN))

```

Single Feature Model Metrics

Accuracy: 0.5467128027662743 Precision: 0.2535211267569926 Recall:
0.18749999999804687
F1 Score: 0.21556886178407256

```

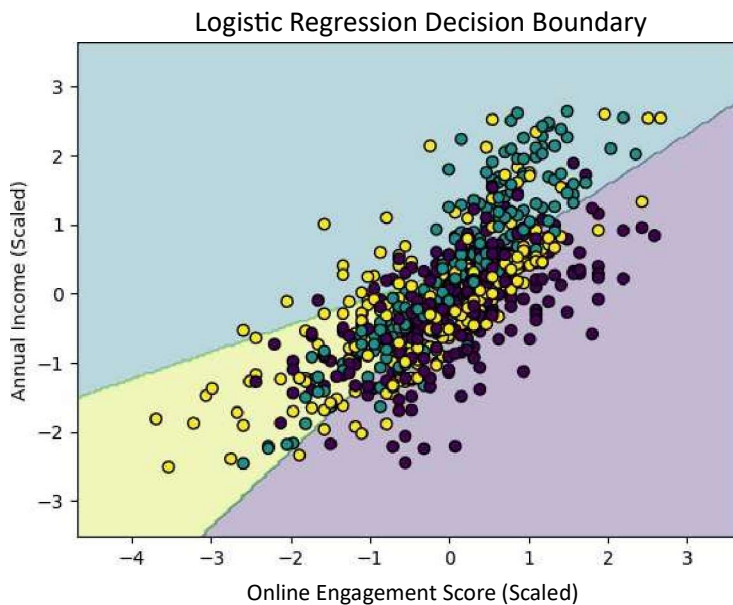
x_min, x_max = X_train[:, 0].min(), X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min(), X_train[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)

Z = model_multi.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z,
alpha=0.3)

plt.scatter(X_train[:, 0], X_train[:, 1], c=Y_train, edgecolors=
plt.xlabel("Online Engagement Score (Scaled)")
plt.ylabel("Annual Income (Scaled)")
plt.title("Logistic Regression Decision Boundary")

plt.show()

```



```

A_range = np.linspace(A_train.min(), A_train.max(), 300).reshape(-1, 1)

B_prob = model_single.predict_proba(A_range)[:, 1]

plt.scatter(A_train, B_train, color='blue', label="Actual Data")

plt.plot(A_range, B_prob, color='red', linewidth=2, label="Sigmoid Curve")

plt.xlabel("Online Engagement Score (Scaled)")
plt.ylabel("Probability of Customer Response")
plt.title("Logistic Regression (Single Feature)")
plt.legend()

plt.show()

```

Logistic Regression (Single Feature)

